Java

Zuletzt aktualisiert am: 21.10.2026

Inhaltsverzeichnis

1. Die `for`-Schleife 1
2. Datenlisten 2
   2.1. Arrays 2
   2.2. Referenztypen 5
3. Ein neues Programmierparadigma 6
   3.1. Funktionen und Methoden 6
4. Aufrufen einer Funktion 6
   4.1. Die Rückgabe einer Funktion 7
   4.2. Funktionen ohne Rückgabe 7
5. Schreiben einer eigenen Funktion 8
   5.1. Der Funktionskopf 8
   5.2. Der Funktionskörper 9
   5.3. Funktionen platzieren und ausführen 9
6. Die Main-Methode 10
7. Vorteile von Funktionen 11
8. Konvertierung 12
   8.1. Implizite Konvertierung 12
   8.2. Explizite Konvertierung und Casting 13
   8.3. Parsing 15
   8.4. Zusammenfassung 15

1. Die `for`-Schleife

Folgende Schleife:

```java
int i = 0;
while (i < 10)
{
    // ...
    i++;
}
```

... kann auch so geschrieben werden:

```java
for (int i = 0; i < 10; i++)
{
    // ...
}
```

Drei Ausdrücke werden in den Schleifenkopf geschrieben: `for (A; B; C)`:

A: Start Dieser Ausdruck wird **nur einmal** vor Beginn des ersten Durchgangs ausgeführt. Meistens wird hier eine Zählvariable initialisiert.

B: Bedingung Hier steht der boolesche Ausdruck für die Abbruchbedingung, wie Sie Ihn schon aus der `while`-Schleife kennen: Sobald die Bedingung `false` ist, wird die Schleife beendet.

C: Update Dieser Ausdruck wird immer am Ende eines Durchganges ausgeführt. Hier wird normalerweise die Zählvariable (aus A) hoch/runter gezählt. Diesen Ausdrück könnten Sie auch als letzten Befehl in den Schleifenkörper schreiben. Er steht aber oben mit im Schleifenkopf für bessere Lesbarkeit.

Die `for`-Schleife wird fast immer wie im oberen Beispiel verwendet. Sie können aber beliebige Ausdrücke verwenden, solange die Syntax es erlaubt:

```java
for (double piePiece = 1.0; piePiece > 0.1; piePiece = piePiece / 2.0)
{
    IO.println("Kuchenstück: " + piePiece);
}
```

Alle drei Ausdrück sind optional und können auch leer gelassen werden. In diesem Beispiel wurden sogar alle drei Ausdrücke weggelassen:

```java
// Endlosschleife mit FOR
for(;;) // <-- Alle drei Ausdrücke wurden weggelassen
{
    IO.println("Ist Mittagessen fertig?");
}

// Endlosschleife mit WHILE
while(true)
{
    IO.println("Ist Mittagessen fertig?");
}
```

2. Datenlisten

Eine Variable ist wie eine Schachtel, die einen Wert mit einem bestimmten Datentypen enthält. Der Name der Variable (der *Bezeicher*) ist das Label auf der Schachtel und der Datentyp bestimmt die Größe der Schachtel.

java
```java
String article = "Mehl";
```

Alle unsere Schachteln liegen im Hauptspeicher des Systems und unser Programm findet die Werte mit denen wir arbeiten wollen anhand der Labels auf den Schachteln wieder.

Wie arbeiten wir aber mit Auflistungen von Werten? Wenn wir eine Einkaufsliste mit vier unterschiedlichen Artikeln darstellen wollen, dann könnten wir vier String-Variablen deklarieren und nummerieren:

java
```java
String article_1 = "Mehl";
String article_2 = "Salz";
String article_3 = "Hot Wheels® Looping Set";
String article_4 = "Hefe";
```

Für jeden Artikel eine Variable anzulegen wird schnell sehr unhandlich. Versuchen Sie mal den User nach einer Zahl zu fragen und anschließend den entsprechenden Artikel anhand der Position in der Einkaufsliste in der Konsole auszugeben (wir ignorieren Mal fehlerhafte Eingaben).

java
```java
String num_str = IO.readln("Wählen Sie einen von vier Artikeln aus (1 bis 4): ");
int num        = Integer.parseInt(num_str);

if (num == 1) { IO.println(article_1); }
if (num == 2) { IO.println(article_2); }
if (num == 3) { IO.println(article_3); }
if (num == 4) { IO.println(article_4); }
```

Eine Schleife kann uns an dieser Stelle auch nicht weiterhelfen. Dazu kommt, dass unsere Lösung nicht mit hunderten Artikeln gut skaliert.

Und was ist, wenn wir den User neue Artikel hinzufügen lassen möchten? Wir können in Java keine neuen Variablen *zur Laufzeit* der Software deklarieren. Alle unsere Variablen müssen (in Java) vor Auslieferung der Software im Quellcode schon deklariert sein.

Aufbauend auf unseren primitiven Datentypen (und `String`) lernen wir jetzt unsere erste sog. **Datenstruktur** kennen.

2.1. Arrays

Die simpelste Datenstruktur ist das **Array** und existiert in den meisten Programmiersprachen. Wir können eine Liste an Werten in einem Array speichern, wenn:

1. alle Werte den **gleichen Datentypen** haben und
2. wir **vorher** wissen **wie viele** Elemente das Array hat.

Stellen Sie sich ein Array vor wie ein Regal. Sie bauen (deklarieren) ein Regal mit `x` Fächern. Jedes Fach hat die gleiche Größe und die Anzahl der Fächer im Regal ändert sich nicht.

java
```
String[] myShelf = new String[5];
```

Anstatt neue Variablen für jeden Wert anzulegen, können wir in unserem Array unsere Werte in der »Regalfachnummer« ablegen.

Aber Vorsicht: Die Fachnummer (der sog. **Index**) beginnt bei `0`!




```
myShelf[0] = "Putzmittel";
myShelf[1] = "Drucker";
myShelf[2] = "alter Laptop";
myShelf[3] = "Akkubohrer";
myShelf[4] = "Kabelkiste";
```

java
```
String[] myShelf = new String[5]; // Deklaration*
myShelf[0] = "Putzmittel";        // Initialisierung
myShelf[1] = "Drucker";           // Initialisierung
myShelf[2] = "alter Laptop";      // Initialisierung
myShelf[3] = "Akkubohrer";        // Initialisierung
myShelf[4] = "Kabelkiste";        // Initialisierung

String index_str = IO.readln("Wählen Sie einen von fünf Fächern aus (1 bis 5): ");
int index        = Integer.parseInt(index_str);

// Index um 1 reduzieren, weil wir Informatiker, anders als der User,
// bei 0 anfangen zu zählen.
index--;
IO.println(myShelf[index]);
```

In diesem Array können wir fünf Strings speichern und auch *nur Strings* und *nur genau fünf* davon.

Ein Array ist nur eine Datenstruktur. In Kombination mit einem *inneren* Datentyp wird ein neuer Datentyp daraus.. Hier sind vier Beispiele aufgelistet:

java
```
int[]     numArr       = new int[1000];
byte[]    streamBuffer = new byte[1024];
boolean[] myBools      = new boolean[4];
String[]  weekDays     = new String[7];
```

Tatsächlich können Sie mit allen Datentypen, die Sie in Zukunft kennenlernen werden, eine Array-Variante deklarieren, indem Sie `[]` hinter den Datentypen schreiben.

Schlüsselwort: **new**

Arrays sind unsere nächsten komplexen Datentypen, neben String, die wir kennenlernen. Genau wie Strings sind Arrays Objekte. Objekte werden mit dem `new`-Schlüsselwort *initialisiert*:

java
```java
String[] myShelf;         // Deklaration
myShelf = new String[5]; // Initialisierung
```

Bei der Initialisierung von Strings reichen die doppelten Anführungsstriche. Die Syntax mit dem `new`-Schlüsselwort funktioniert aber auch:

java
```java
String name1 = "Herbert";
String name2 = new String("Herbert");
```

Dieses Schlüsselwort gehört zur *Objektorientierten Programmierung* und wird dort weiter vertieft.

Standardinitialisierung

Interessant ist folgendes Beispiel:

java
```java
int[] myNumberArray = new int[5];
IO.println(myNumberArray[3]);
```

Funktioniert das? Was wird hier ausgegeben?

Kurzschreibweise

Auch ein Array kann mit nur einem Befehl deklariert und initialisiert werden:

java
```java
double[] myGrades = {6.0, 6.0, 6.0, 5.9, 6.0};
IO.println(myGrades[3]); // Ausgabe: 5.9
```

Arrays sind eine sehr simple aber auch unflexible Datenstruktur. Sobald die Größe festgelegt ist, können keine Elemente mehr hinzugefügt werden und wir können auch keine Datentypen mischen. Neben dem Array werden wir in Zukunft noch einige andere Datenstrukturen kennenlernen und sogar eigene programmieren. Diese Datenstrukturen arbeiten aber sehr häufig im Hintergrund mit Arrays.

2.2. Referenztypen

Arrays sind auch unser erster *Referenztyp* den wir kennenlernen. Bei unseren primitiven Datentypen wird ein Wert von einer Variable mit dem Zuweisungsoperator in die andere kopiert:

java
```
double temperature    = 19.3;
double temperatureNew = temperature;

temperatureNew += 0.5;

IO.println("Die Temperatur ist von " + temperature
           + " auf " + temperatureNew + " gestiegen.");
```

Betrachten Sie im Vergleich dazu folgendes Beispiel. Testen Sie es in Ihrer IDE aus:

java
```
// Erstes Regal definieren
String[] myFirstShelf = new String[3];
myFirstShelf[0] = "Putzmittel";
myFirstShelf[1] = "Drucker";
myFirstShelf[2] = "alter Laptop";

// Regal "kopieren"
String[] mySecondShelf = myFirstShelf;
mySecondShelf[2] = "neuer Laptop"; // Ein Element im neuen Regal ändern

// Drittes Fach von beiden Regalen ausgeben
IO.println("Originales Regal (Fach 3): " + myFirstShelf[2]);
IO.println("Kopiertes Regal (Fach 3): " + mySecondShelf[2]);
```

Wenn Sie Ihre Ausgabe betrachten und weiter testen, werden Sie feststellen, dass immer noch nur *ein* Array im Speicher existiert. Nur haben Sie jetzt zwei Variablen, die das gleiche Array **referenzieren**. Diese Eigenschaft wird Ihnen beim Programmieren noch sehr oft begegnen. Komplexe Datentypen sind fast immer **Referenztypen**.

So *klonen* Sie ein Array übrigens richtig, wenn Sie eine unabhängige Kopie brauchen:

java
```
String[] mySecondShelf = myFirstShelf.clone();
```

Aus genau diesem Grund vermeiden wir den Vergleichsoperator `==`, wenn wir Strings miteinander vergleichen. In dem Fall werden Referenzen und nicht Werte miteinander verglichen. Das gleiche Problem haben Sie auch mit Arrays. Um bei dem Beispiel mit den beiden Regalen zu bleiben, müssen Sie tatsächlich jedes »Regalfach« einzeln mit dem anderen Regal vergleichen. Nur so können Sie den Inhalt zweier Arrays miteinander vergleichen: Element für Element (z. B. in einer Schleife).

Arraylänge

Ähnlich wie mit Strings können Sie auch an die Arraylänge herankommen.

java
```
IO.println(someArray.length);    // Keine runden Klammern nach .length
IO.println(someString.length()); // Runde Klammern nicht vergessen!
```

Nützlich um Arrays mit einer Schleife zu durchlaufen!

3. Ein neues Programmierparadigma

Im Grunde haben wir einen Großteil der *Imperativen Programmierung* in Java behandelt. Sie können Operationen mit Daten ausführen und diese in Programmabzweigungen und Schleifen in komplexe Software verwandeln. Sie haben mit Arrays auch schon eine Datenstruktur kennengelernt. Theoretisch könnten Sie mit diesem Wissen jede beliebige Software schreiben. In der Praxis ist es aber nicht ratsam ein Hochhaus nur mit einer Schippe und einer Schubkarre zu bauen.

Mit Funktionen steigen wir in die sog. *Prozedurale Programmierung* ein, und nähern uns damit einen großen Schritt an die *Objektorientierte Programmierung* (OOP) heran. Funktionen sind unser erstes Werkzeug mit dem wir unseren Code strukturieren und optimieren.

3.1. Funktionen und Methoden

Funktionen sind im Kern **wiederverwendbare Code-Blöcke.**

Eventuell ist Ihnen schon aufgefallen, dass Sie Code für eine vorherige Problemlösung nahezu unverändert für Ihr aktuelles Problem, an dem Sie gerade vielleicht sitzen, kopieren und anpassen können. Oder Ihre Programmlogik ist so lang und komplex geworden, dass Sie Ihren Code gerne aufteilen und innerhalb Ihrer Java-Datei nach Ihren eigenen Wünschen ordnen könnten, ohne die tatsächliche Reihenfolge der Instruktionen zu verändern.

Mit Funktionen können wir u. a. diese beiden Situationen adressieren.

Fast synonym wird für Funktionen im allgemeinen Sprachgebrauch auch der Begriff **Methode** verwendet. Bis jetzt haben wir versucht so gut es geht das Thema OOP zu vermeiden, was in Java besonders schwierig ist. Wir sind aber schon nah dran, den Vorhang zu lüften!

Methoden sind nämlich Funktionen *in Objekten.* **Jede Methode ist eine Funktion aber nicht jede Funktion ist eine Methode.** Einige Methoden haben Sie schon im Unterricht verwendet.

```java
String myName  = "Walter White";
int nameLength = myName.length(); // length() ist die Methode!
```

`myName` ist das Objekt, auf dem wir die Methode `length()` **aufrufen**. Der »wiederverwendbare Code-Block« hinter `length()` ist irgendwo in Javas sog. *Standardbibliothek* versteckt.[1]

Wir werden aber noch ein letztes Mal um OOP herumtanzen und den Objekt-Part später vertiefen. Bis dahin werden wir auch auf die Differenzierung zwischen Funktion und Methode verzichten und vereinfacht immer von Funktionen sprechen, auch wenn *Methode* korrekter wäre.

4. Aufrufen einer Funktion

Mit Funktionen kommen wir auf zwei Arten in Berührung:

1. Wir **rufen** vorhandene Funktionen **auf**.[2] Das können Funktionen sein, die uns Java oder andere Entwickler zur Verfügung stellen, z. B. *String-Methoden* wie `.length()`, `.isEmpty()` und `.equals()`.
2. oder wir schreiben unsere eigenen Funktionen!

Schauen wir uns ersteinmal den *Aufruf* aus dem vorherigen Beispiel genauer an, bevor wir unsere eigenen Funktionen definieren:

[1] In IntelliJ können Sie mit `Strg` + `Rechtsklick` auf den Funktionsnamen zum Code-Block springen.
[2] Auf Englisch: ***calling*** *a function/method*

java
```java
String myName  = "Walter White";
int nameLength = myName.length();
```

Funktionsaufrufe sind auch **Ausdrücke** wie `1 + 2` die einen Wert produzieren den wir z. B. in einer Variable speichern können.

Genau wie jeder Ausdruck hat auch eine Funktion einen Datentyp. Man spricht von **Rückgabewerten** und **Rückgabetypen**, weil sie von der Funktion *zurückgegeben* werden. Nachdem der Code-Block von `length()` ausgeführt wurde, erhalten wir als **Rückgabe** die Länge des Strings. Dieser Wert ist vom Datentyp `int`, also weisen wir diesen Wert einer Variable `nameLength` vom passenden Datentyp zu.

Überall dort, wo Sie bisher Ausdrücke verwendet haben (z. B. in `if`-Statements, Schleifen oder rechts vom `=` Zuweisungsoperator) können Sie auch Funktionen aufrufen. Es muss nur der Rückgabetyp passen.

4.1. Die Rückgabe einer Funktion

Jede Funktion/Methode *kann* einen Rückgabewert haben. Dies ist quasi das *Rechenergebnis* der Methode, das sie zur weiteren Verwendung im Code zur Verfügung stellt. Rufen wir eine Methode auf, müssen wir genau wissen, welchen Rückgabetyp sie hat – nur so können wir ihr Ergebnis z. B. in einer Variable speichern:

java
```java
String myName = "Walter White";

int nameLength  = myName.length();              // Rückgabetyp: int
boolean isEmpty = myName.isEmpty();             // Rückgabetyp: boolean
boolean equal   = myName.equals("Heisenberg");  // Rückgabetyp: boolean
char firstChar  = myName.charAt(0);             // Rückgabetyp: char
```

Die Methode `.equals()` nimmt dazu noch einen **Parameter** als **Eingabe** entgegen und hat damit den klassischen Ablauf einer Funktion: Eingabe → Verarbeitung → Rückgabe

> **Wer bestimmt den Rückgabetyp?** 📖
>
> Der Rückgabetyp einer Funktion wird bei ihrer Definition im *Funktionskopf* festgelegt (auch *Funktionssignatur* genannt). Dazu mehr im nächsten Abschnitt.

4.2. Funktionen ohne Rückgabe

Eine Funktion/Methode **muss** keinen Rückgabetyp haben. In diesem Fall gibt es kein *Rechenergebnis*, das man anschließend einer Variable zuweisen könnte. Man ruft die Methode dann ohne Zuweisung auf:

java
```java
IO.println("Hi!");                // funktioniert
String result = IO.println("Hi!"); // COMPILER FEHLER
```

Das heißt jedoch nicht, dass diese Methode nichts macht. Sie stellt nur kein Ergebnis zur Weiterverwendung zur Verfügung. Im Fall von `IO.println()` wird Text im Terminal ausgegeben. Den Text nochmal als `String` zurückzugeben wäre nicht unbedingt nützlich.

Parameter und Rückgaben sind also **optional**.

Übung 1

Initialisieren Sie einen beliebigen String.
Finden Sie heraus, wie Sie die folgenden Methoden auf dem String aufrufen, und weisen Sie das Ergebnis der Methoden einer Variable vom passenden Datentyp zu:

```
1. someStr.toUpperCase()
2. someStr.toCharArray()
3. someStr.contains()
```

5. Schreiben einer eigenen Funktion

Funktionen schreibt man, indem man sie **definiert**. Die Definition bestimmt das gesamte Verhalten der Funktion: Die Eingabe, die Verarbeitung, und die Rückgabe.

Betrachten wir dazu direkt ein Beispiel:

```java
boolean isAdult(int age) // Funktionskopf
{
    return age >= 18;    // Funktionskörper
}
```

Die Funktion `isAdult()` überprüft ein gegebenes Alter auf Volljährigkeit. Das wäre z. B. eine sinnvolle Funktion, wenn wir in unserem Code sehr häufig prüfen müssen, ob jemand volljährig ist.

Die Definition der Funktion besteht aus **Funktionskopf** und **Funktionskörper**:

- Im **Funktionskopf** (auch *Funktionssignatur* genannt) werden der Name der Funktion und weitere wichtige Eigenschaften (Rückgabetyp, Parameter, …) definiert. Anhand des Funktionskopfes lässt sich ablesen, wie man die Funktion später aufzurufen hat.
- Im **Funktionskörper** steht der eigentliche Code, der beim Aufruf der Funktion ausgeführt wird. Man spricht auch vom Code, der sich *in der Funktion* befindet.

5.1. Der Funktionskopf

Im **Funktionskopf** werden einige allgemeine Eigenschaften definiert:

- Der **Name** der Funktion: `isAdult`
  - Anhand ihres Namens können wir die Funktion später an anderer Stelle aufrufen.
- Der **Rückgabetyp** der Funktion: `boolean`
  - Soll eine Funktion keinen Rückgabe haben, schreibt man hier **`void`**
- Die **Parameter**, die die Funktion **entgegennimmt**: `int age`
  - Ein Parameter ist eine Variable, die die Funktion benötigt, um richtig zu funktionieren: Zusätzliche Informationen oder sonstige **Eingaben**, die die Funktion anschließend verarbeitet
  - Zu jedem Parameter muss angegeben werden, welchen Typ er haben soll
  - Eine Funktion kann null `()` bis beliebig viele Parameter entgegennehmen, die dann durch Kommas in den Klammern getrennt werden
  - **Erst beim Aufruf der Funktion wird festgelegt, welchen tatsächlichen Wert der Parameter hat.** Auf diese Weise können wir die Funktion mehrfach mit unterschiedlichen Eingabedaten aufrufen.

Die Komponenten des Funktionskopfes stehen immer in der gleichen Reihenfolge:

```
Rückgabetyp Funktionsname(Typ Parameter 1, Typ Parameter 2, ...)
```

Namenskonventionen bei Funktionen 📖

Der Name von Funktionen wird genau wie Variablen per Konvention in *camelCase* geschrieben, d.h. **der erste Buchstabe ist immer klein**. Achten Sie auf aussagekräftige Namen ihrer Methoden: Sie enthalten häufig ein Verb, das beschreibt, was die Methode tut.

5.2. Der Funktionskörper

Im Funktionskörper steht der eigentliche Code, der ausgeführt wird, wenn wir die Funktion später aufrufen.
In unserem Beispiel ist das nur eine Zeile Code: `return age >= 18;` .

Hier lernen wir das Stichwort **`return`** (engl. für »zurückgeben«) kennen:

- Hinter `return` steht der **Rückgabewert** der Methode: Diese Methode gibt die Info zurück, ob der Parameter `age` größer oder gleich 18 ist.
- Der Rückgabewert muss dem Rückgabetypen im Funktionskopf entsprechen: Wenn wir eine Ja/Nein-Info mit `return` zurückgeben wollen, müssen wir im Funktionskopf als Rückgabetyp `boolean` angeben.
- Mit dem `return`-Statement wird die Funktion **automatisch beendet**. Code, der hinter dem return-Statement steht, wird nicht mehr ausgeführt!

5.3. Funktionen platzieren und ausführen

Funktionen müssen an der richtigen Stelle im Code definiert werden, damit man sie aufrufen kann. Zum ersten Mal platzieren wir hier etwas außerhalb der Main-Methode.

Eine Datei mit `main`-Methode und der Funktion `isAdult()` würde folgendermaßen aussehen:

java

```java
// Klassische Main-Methode:
void main()
{
    int myAge = 25;
    boolean adult = isAdult(myAge);   // Aufruf der Funktion isAdult()
}

// Definition der Funktion isAdult():
boolean isAdult(int age)
{
    return age >= 18;
}
```

Achten Sie auf die Platzierung der geschweiften Klammern: Zunächst muss die Main-Methode mit einer schließenden geschweiften Klammer beendet werden, bevor wir die nächsten Funktion definieren.

In Zeile 4 wird die Funktion dann aufgerufen, damit sie auch ausgeführt wird. Diese Funktion wird (im Gegensatz zu String-Methoden) nicht auf einem Objekt mit der Punktschreibweise aufgerufen, sondern kann alleine für sich aufgerufen werden.

Reihenfolge von Funktionsdefinitionen

Grundsätzlich ist die Reihenfolge, in der Funktionen in einer Datei definiert werden, egal. Wir könnten also auch zuerst die Funktion `isAdult()` schreiben und darunter die Main-Methode platzieren. Im Zusammenhang mit *Klassendefinitionen* gibt es Konventionen für die Reihenfolge, die wir später kennenlernen.

Übung 2

Definieren Sie eine weitere Funktion, die zwei `int`-Zahlen miteinander addieren soll. Der **Funktionskopf** ist Ihnen schon vorgegeben:
`int add(int number1, int number)`
Schreiben Sie nun den Funktionskörper (d.h. das tatsächliche Addieren der Zahlen mit Rückgabe des Ergebnisses) und platzieren Sie die Funktion an richtiger Stelle in Ihrer Datei.
Rufen Sie die Funktion anschließend in der Main-Methode mit den Werten `13` und `47` auf und weisen Sie das Ergebnis einer passenden Variable zu.

Übung 3

Definieren Sie eine Funktion `calcPercent()` (engl. für »Berechne Prozent«), die zwei Eingaben (d.h. Parameter) entgegennimmt:
- Eine Zahl (`double`)
- Einen Prozentsatz (`int`)

Die Funktion soll dann den angegebenen Prozentsatz der übergebenen Zahl berechnen und das Ergebnis zurückgeben.
Beispiel: Für die Eingaben `200.0` und `25` soll das Ergebnis `50.0` herauskommen, weil dies 25% von 200 entspricht.

Rufen Sie die Funktion anschließend zweimal mit folgenden Eingaben in der Main-Methode auf und weisen Sie die Ergebnisse je einer passenden Variable zu:
- `300.0` und `50` (gewünschtes Ergebnis: `150.0`)
- `222.2` und `20` (gewünschtes Ergebnis: `44.44`)

6. Die Main-Methode

Die Main-Methode ist auch nichts weiter als eine Funktion in Java, die von uns definiert wird. Sie hat jedoch ein paar besondere Eigenschaften.

Die Main-Methode hat niemals eine Rückgabe. Daher schreiben wir vor die Definition der Funktion das Stichwort `void` :

java

```java
// Main-Methode mit Rückgabetyp (FUNKTIONIERT NICHT):
int main()
{
    // ...
}
```

```java
// Main-Methode ohne Rückgabetyp:
void main()
{
    // ...
}
```

Wird ein Java-Programm ausgeführt, wird im Standard-Fall nach einer Funktion mit dem Namen `main()` gesucht, die dann ausgeführt wird. Wir müssen sie also im Gegensatz zu allen anderen Funktionen nicht explizit aufrufen.

7. Vorteile von Funktionen

Für Anfänger erscheint es manchmal mühsam, Code in eine Funktion auszulagern, die dann extra aufgerufen werden muss.

Mit größer werdenden Projekten ist es jedoch sehr sinnvoll, seinen eigenen Code in Funktionen bzw. Methoden zu strukturieren, um sog. **Spaghetti-Code** zu vermeiden.

Vorteile von Funktionen:

- Code-Blöcke werden dadurch **wiederverwendbar**, Code-Duplizierung wird drastisch reduziert
  - Daraus folgt eine bessere **Wartbarkeit** des Codes, da weniger Stellen angepasst werden müssen
  - Daraus folgt eine **reduzierte Fehleranfälligkeit**
- Derselbe Code kann **mit verschiedenen Eingabedaten ausgeführt werden**, ohne den Code selbst anpassen zu müssen
- **Code-Testing** wird vereinfacht, da man einzelne Funktionen getrennt voneinander prüfen kann
- Funktionen mit sinnvoll gewählten Namen helfen, den Code **visuell zu strukturieren** und für Andere nachvollziehbar zu machen

Übung 4



Öffnen Sie eine Ihrer vergangenen Programmdateien (z. B. die Datei `_03_ArrayGrades/GradeCalc.java`) und führen Sie folgende Schritte durch:

1. Identifizieren Sie eine gekapselte Funktionalität im Code (z. B. das Berechnen der Summe eines `int`-Arrays).
2. Lagern Sie diese Funktionalität in eine Funktion aus, der Sie einen aussagekräftigen Namen geben. Ermitteln Sie hierzu:
   - was die **Eingabe** der Funktion ist (d.h. die Parameter, die die Funktion braucht, um ihr Ziel zu erreichen)
   - was die **Rückgabe** sein soll (d.h. das Ergebnis, das die Funktion dem restlichen Programm zur Verfügung stellen soll)
3. Rufen Sie die Funktion anschließend dort auf, wo Sie sie benötigen.

8. Konvertierung

Bevor wir in das Thema OOP eintauchen, noch ein paar Worte zur Konvertierung von primitiven Datentypen in Java. Dazu sind einige Begriffserklärungen und Konzepte notwendig. Ein Konzept haben Sie schon vermutlich angewandt, ohne es zu merken.

8.1. Implizite Konvertierung

Variablen können ihren Datentypen nicht verändern aber Werte können durchaus zwischen Datentypen hin und her konvertiert werden.

```java
byte  a = 5;  // Integer Literal zu Byte => 5
int   b = a;  // Byte zu Integer         => 5
float c = b;  // Integer zu Float        => 5.0
IO.println(c) // Float zu String         => "5.0"

// "Integer / Integer" ergibt einen Integer => 3
double d = 7 / 2; // Integer 3 wird dann zu Double 3.0

// "Double / Integer" wird zu "Double / Double" und ergibt einen Double => 3.5
double e = 7.0 / 2; // Das Ergebnis des Ausdrucks muss nicht konvertiert werden
```

Diese Konvertierungen sind alle *implizit*. Java nimmt automatisch eine Konvertierung vor, je nach Kontext. Z. B. wenn ein Wert von einer Variable in eine andere kopiert wird oder zwei unterschiedliche Typen in einem arithmetischen Ausdruck vorkommen. Auch Funktionsargumente und Rückgabewerte werden bei Funktionsaufrufen oft implizit konvertiert (wie im `IO.println()` Beispiel oben). Es ist leider ein Irrglaube, dass implizite Konvertierungen *verlustfrei* sind. Die Konvertierung von `int` zu `float` kann in der IEEE-754 wie schon erwähnt zu Fehlern in der Darstellung führen. Hier ein Beispiel:

```java
int a = 16_777_217;
int b = 16_777_216;
float x = a; // Konvertiert implizit zu: 16_777_216
float y = b; //                                    ^
IO.println(x); // 1.6777216E7 (E7 => Komma um 7 Stellen nach rechts verschieben)
IO.println(y); // 1.6777216E7
IO.println(x == y); // true
```

Implizite Konvertierungen zeichnen sich allein dadurch aus, dass Sie nicht *explizit* in den Java-Quelltext von Ihnen hingeschrieben werden müssen. Folgende Datentypen lassen sich meistens implizit in »kleinere« oder gleich große Datentypen konvertieren:

Quelldatentyp	Zieldatentyp
`byte`	`short`, `int`, `long`, `float`, `double`
`short`	`int`, `long`, `float`, `double`
`char`	`int`, `long`, `float`, `double`
`int`	`long`, `float`, `double`
`long`	`float`, `double`
`float`	`double`

Tabelle 1: Implizite Konvertierungen in Java

Type Promotion 📖

Für ganzzahlige Typen die kleiner sind als `int` werden arithmetische Ausdrücke von Java automatisch zu einem `int`. Noch ein Grund `int` als Standarddatentypen für Ganzzahlen zu verwenden, auch wenn Sie nicht den ganzen Wertebereich benötigen.

java
```
  byte a = 1;
  byte b = 2;
//byte c = a + b; => ERROR: Ausdruck wird zu einem int "befördert"
  byte c = (byte) (a + b); // funktioniert
```

8.2. Explizite Konvertierung und Casting

Das sog. *type widening* ist (meistens) verlustfrei und oft implizit möglich. Java wird Sie aber zur *expliziten* Konvertierung auffordern, wenn Sie bspw. einen Wert aus einem großen Datentypen wie `int` in einen kleineren Datentypen wie `byte` schreiben wollen (*type narrowing*). Als Daumenregel gilt, dass man Werte *explizit* konvertieren muss, wenn **Informationen verloren gehen *könnten***.

Dieser Code kompiliert nicht. Auch wenn die Zahl 1 Problemlos in alle numerischen Datentypen passt.

java
```
int  a = 1;
byte b = a; // ERROR
```

Der Compiler achtet selten auf die Werte in Ihrem Quelltext. Eventuell könnte die 1 ja auch von einer Usereingabe kommen und steht damit erst zur Laufzeit (runtime) fest. An stellen im Code, bei denen Sie als Entwickler wissen, dass so eine Konvertierung sicher ist (weil Sie vielleicht einen gültigen Wertebereich mit `if`-Statements vorher sicherstellen) können Sie die Konvertierung *explizit* machen. Das geht am einfachsten mit dem **Cast-Operator.**

java
```
int  a = 1;
byte b = (byte) a; // Cast-Operator o. a. Casting
```

Schreiben Sie einen beliebigen *Zieldatentypen* vor Ihren Ausdruck in runden Klammern. Sie sind aber selbst in der Verantwortung, dass die Konvertierung keine Fehler verursacht. Wenn Sie Fließkommazahlen in Ganzzahlen umwandeln, dann verlieren Sie z. B. die Nachkommastellen (ohne Rundung):

java
```
// Nachkommastellen gehen beim Casting verloren
int a = (int)  4.9; //  4
int b = (int)  4.1; //  4
int c = (int) -4.9; // -4
int d = (int) -4.1; // -4

// Zu große Zahlen werden auf den Grenzwert des Datentyps gesetzt
int e = (int)  3.9e20; //  2147483647
int f = (int) -3.9e20; // -2147483648
// Hinweis: "e20" => Wissenschaftliche Notation
//                => Komma um 20 Stellen nach rechts verschieben
```

Hier sind vielleicht Funktionen aus Javas integrierter `Math`-Bibliothek hilfreicher:

```java
double num = -4.5;

IO.println((int)num);       // -4
IO.println(Math.round(num)); // -5
IO.println(Math.floor(num)); // -5.0
IO.println(Math.ceil(num));  // -4.0
```

Wichtig ist auch, dass Casting keine Wertebereiche überprüft. Wenn Sie einen Integer zu einem Byte umwandeln, führt das schnell zu Unfällen, die der Casting-Operator nicht für Sie verhindert! So wird aus einer `131` (Integer) bei der Umwandlung zu Byte eine `-125` (probieren Sie es mal aus). Dass die größte repräsentierbare Zahl in einem Byte die `127` ist, ist ein Hinweis auf das Problem. Ein Blick in die binäre Representation zeigt die genaue Ursache:

```java
// Tipp: mit 0b als Präfix können Sie Binärzahlen schreiben

int numInt  =  0b00000000_00000000_00000000_10000011; // 1 + 2 + 128 =  131
// Als ein BYTE: --------_--------_--------_10000011 ==> 1 + 2 - 128 = -125
//               ^                         ^
//               Vorzeichen-Bit (int)   Vorzeichen-Bit (byte)

byte numByte = (byte)numInt; // <-- explizite Konvertierung via Casting
IO.println(numInt);  //  131
IO.println(numByte); // -125
```

Die `131` ist nicht darstellbar als Byte. Java wird beim Casting trotzdem alle *niedrigstwertigen* Bits (alle Bits von *rechts* aus) in den Datentyp pressen und alle anderen Bits (links) einfach abschneiden.

Von einem kleinen Ganzzahltypen zu einem großen (z. B. `short` zu `long`) entsteht kein Verlust beim casting. Java muss nur führende Nullen hinzufügen und das Vorzeichenbit verschieben, damit negative Zahlen nicht plötzlich positiv werden. Alle anderen Bits bleiben unberührt.

Randnotiz 📖

Der Cast-Operator ist nicht der einzige Weg explizit zu konvertieren:

```java
int numInt = 42;

String text1 = String.valueOf(numInt);
String text2 = Integer.toString(numInt);

double numDouble = 4.2;
numInt = Double.valueOf(numDouble).intValue(); // umständlich
numInt = (int) numDouble;                      // einfacher

// round() konvertiert z. B. double zu long (mit ZUSÄTZLICHER Rundungslogik!)
long numLong = Math.round(numDouble);

// Kleiner Trick (aber technisch gesehen implizit)
String text3 = "" + numInt;
```

8.3. Parsing

Daten aus Texten (`Strings`) auszulesen ist bei der Konvertierung eine separate Kategorie. Während Bit-Repräsentationen von Zahlen nach eindeutigen Regeln konvertiert werden können, sieht es bei Texten deutlich komplexer aus. Die gleiche Zahl wird in unterschiedlichen Ländern und unterschiedlichen Fachbereichen anders notiert:

- `"1000,5"`
- `"1000.5"`
- `"1.000,5"`
- `"1,000.5"`
- `"1.0005e3"`
- `"1,0005e3"`

Und was ist mit Leerzeichen? Was ist mit Auflistungen von Zahlen? Werden die Zahlen mit `,` oder `.` oder `;` oder Leerzeichen oder Zeilenumbrüchen voneinander getrennt? Hier legt die Programmiersprache die Verantwortung in Ihre Hände aus Texten die Daten korrekt zu *parsen*. Ganz im Stich werden Sie zum Glück nicht gelassen. Einige Parsing-Methoden aus Javas Standardbibliothek haben Sie schon verwendet:

```java
double myDoubleFromText  = Double.parseDouble("13.37");
int myIntFromHexadecimal = Integer.parseInt("FF", 16); // FF (Hex) => 255 (Dez)
```

Im Laufe des Unterrichts werden aber noch kompliziertere Parsing-Probleme auf uns zukommen.

8.4. Zusammenfassung

- Für **jede Typ-Kombination und jeden Kontext** sind bestimmte Regeln für die Konvertierung in der Java-Spezifikation definiert[3] unabhängig von Begriffen wie implizit, explizit, casting etc.
- **Implizite Konvertierung** muss nicht in den Quelltext geschrieben werden und geschieht automatisch.
- **Explizite Konvertierung** muss *explizit* in den Quelltext geschrieben werden. Häufig mit dem Cast-Operator `int a = (int) 2.2` oder mit anderen Konvertierungsmethoden.
- Von kleinen Ganzzahltypen in größere und von `float` zu `double` gehen keine Informationen verloren ⟶ **type widening**
- Zwischen Ganzzahlen und Fließkommazahlen können Informationen verloren gehen (unabhängig von der Größe des Datentypen).
- Von großen Datentypen zu kleineren Datentypen muss explizit konvertiert werden, z. B. via **casting**, und es können Informationen verloren gehen! ⟶ **type narrowing**
- Konvertierungen von Strings zu anderen Datentypen nennt man **parsing** und ist nicht trivial.

[3] https://docs.oracle.com/javase/specs/jls/se26/html/jls-5.html